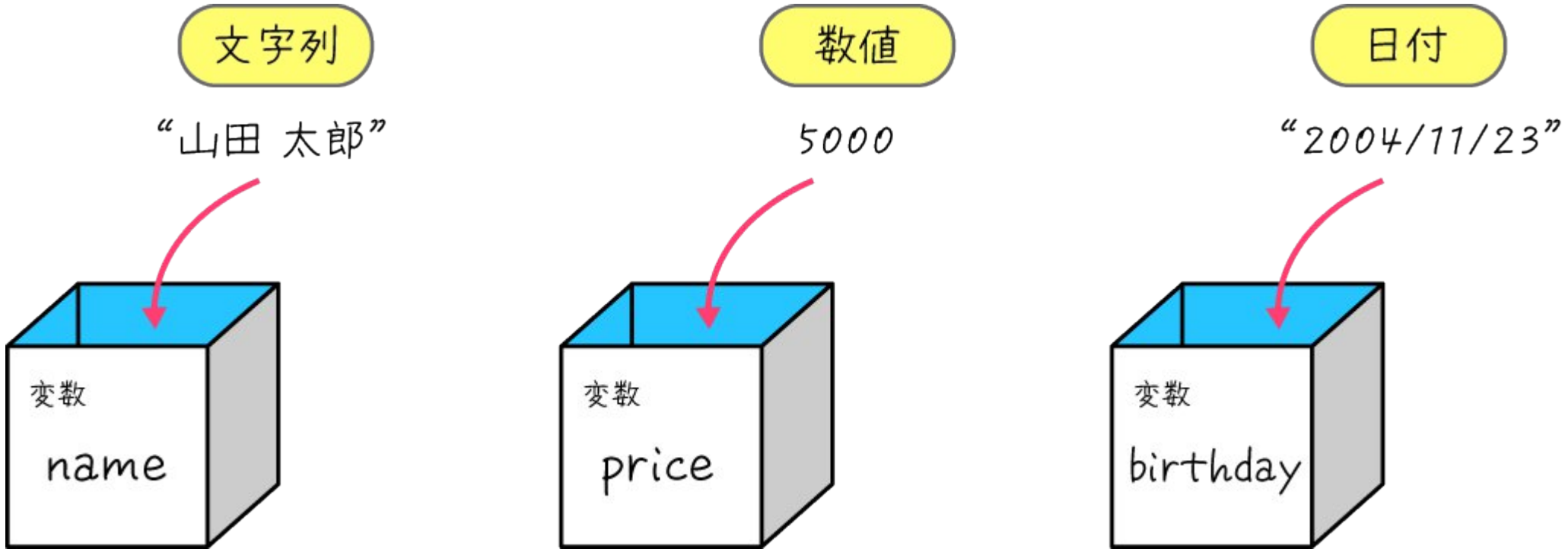
変数・四則演算

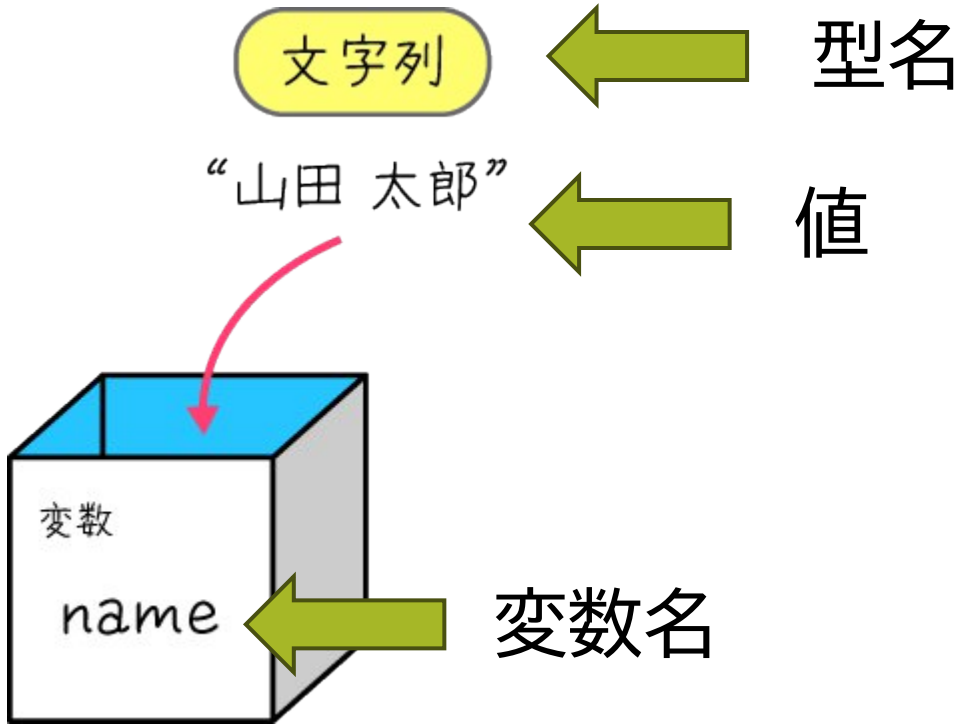
変数とは

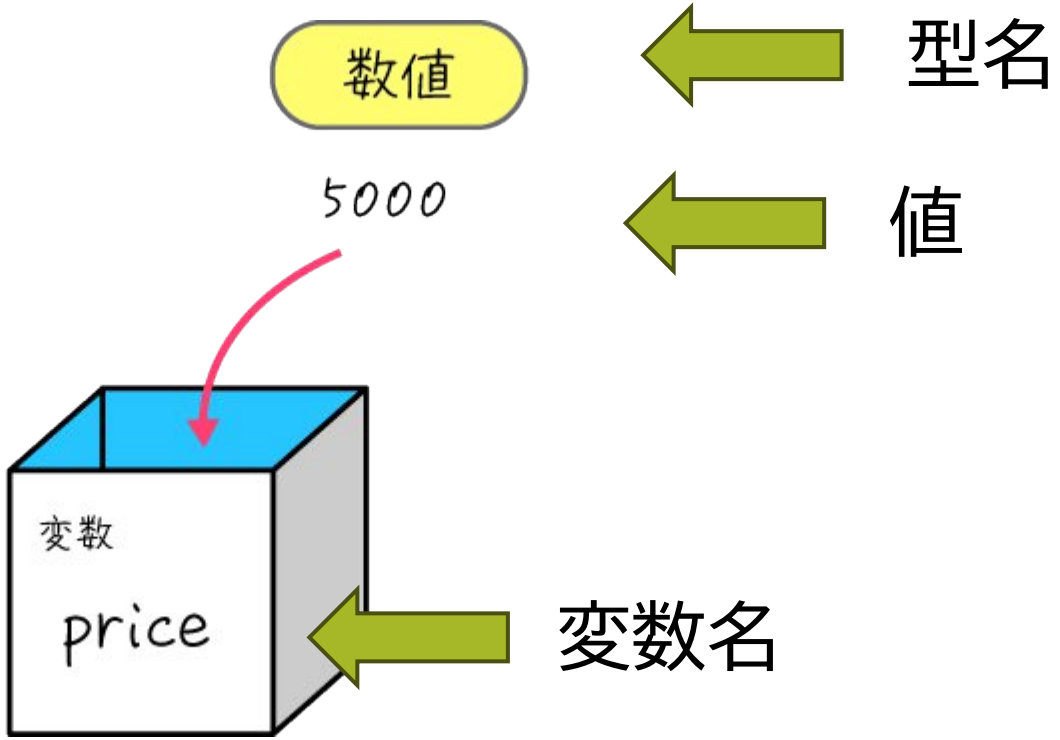
変数とは、数値や文字を格納しておく箱のようなものです。
その箱の中にデータを入れておくと好きな時に、
そのデータを使えるようになります。



変数とは

変数を扱う際には、型名と変数名と値の3つが必要になります。





変数の作成

実際に変数を作成してみましょう。
下記のスクリプトを写してください。

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = "変数 numberに保存されているのは";
        int number = 100;
        Console.WriteLine(text);
        Console.WriteLine(number);
    }

}
```

変数の作成

こんな感じのがでてきたらOKです。

```
変数 numberに保存されているのは
100

D:\001_講師\2026_パンタン授業資料\C言語
セス 8588) は、コード 0 (0x0) で終了しま
デバッグが停止したときに自動的にコンソー
```

変数の作成

最初に説明した、変数の3要素
型名・変数名・値はそれぞれこの部分です。

```
int number = 100;
```

型名　　変数名　　値

変数の型 `int number = 100;`

変数はどんな値を保存したいかによって型を変える必要があります。

```
string text = "変数 numberに保存されているのは";
int number = 100;
```

先程のプログラムでも、
文字列を保存するstring型と整数を保存するint型のように
何を保存するかによって、使い分けています。

変数名 `int number = 100;`

変数名はある程度自由につける事が出来ます。
唯、下記のNG項目があります。

・数字を先頭にできない。
・同じ名前の変数名は使えない（同じスコープ内で）
・予約語と呼ばれるプログラムで別の意味を持っている単語

変数名はプログラマーが自由に決められます。
基本的には、その変数を何に使うかが分かるように変数名をつます。

変数名の悪例

下記のような変数名は避けましょう

```
int number = 100;
```

```
static void Main(string[] args)
{
    string a = "変数 numberに保存されているのは";
    int b = 100;
    Console.WriteLine(a);
    Console.WriteLine(b);
}
```

変数名が明らかに適当なa,bになっています。
これでは、何をするための変数なのか分からないので避けましょう。

変数の値　int number = 100;

変数の値は、事前に決めた型に合ったものでなければいけません。
上の例はint型（整数を入れる箱）に１００を入れています。

```
static void Main(string[] args)
{
    int a = "変数 numberに保存されているのは";
    string b = 100;
    Console.WriteLine(a);
    Console.WriteLine(b);
}
```

試しにint とstringを入れ替えるとこのように赤波線で、エラーが表示されます。

変数の型

変数の型は用途に合わせて複数のものがあります。
必要に応じて使い分けをします。

型名	用途	注釈
bool	Yes,No	Yes→true，No→false
int	整数	上限が21億程度
float	少数	
string	文字列	""の中に文字を書く
	~下記はあまり使わないので知識程度に~	
char	一文字	''の中に文字を書く
double	少数(高精度)	
long	整数(大量)	
longlong	整数(超大量)	
unsigned int	マイナスなしの整数	たまに使う
unsigned float	マイナスなしの少数	たまに使う

変数・四則演算

整数の四則演算

int型は整数を扱うことができる型です。
四則演算(＋,-,*,/)を行うことできます。
また％で計算を行うと、割り算のあまりを求めることができます。

注意

割り算をした結果、答えに少数点がある場合切り捨てされます。

プログラム例

```
1   #include <iostream>
2   #include <string>
3
4   using namespace std;
5
6   void main()
7   {
8       string addText = "足し算の答えは%d¥n";
9       string difText = "引き算の答えは%d¥n";
10      string mulText = "掛け算の答えは%d¥n";
11      string divText = "割り算の答えは%d¥n";
12      string modText = "割り算の余りは%d¥n";
13
14      int firstValue = 6;
15      int secondValue = 5;
16
17      int add = firstValue + secondValue;
18      int dif = firstValue - secondValue;
19      int mul = firstValue * secondValue;
20      int div = firstValue / secondValue;
21      int mod = firstValue % secondValue;
22
23      printf(addText.c_str(), add);
24      printf(difText.c_str(), dif);
25      printf(mulText.c_str(), mul);
26      printf(divText.c_str(), div);
27      printf(modText.c_str(), mod);
28  }
```

実行結果

```
Microsoft Visual Studio デバッグ

足し算の答えは11
引き算の答えは1
掛け算の答えは30
割り算の答えは1
割り算の余りは1

C:\Users\triku\Desktop\C言語基礎\program\Debug\program.exe
デバッグが停止したときに自動的にコンソールを閉じるには、[ツ
動的にコンソールを閉じる] を有効にします。
このウィンドウを閉じるには、任意のキーを押してください...
```

変数・四則演算

小数の四則演算

float型は少数点を含んだ数値を扱うことができる型です。
四則演算(＋,-,*,/)を行うことできます。

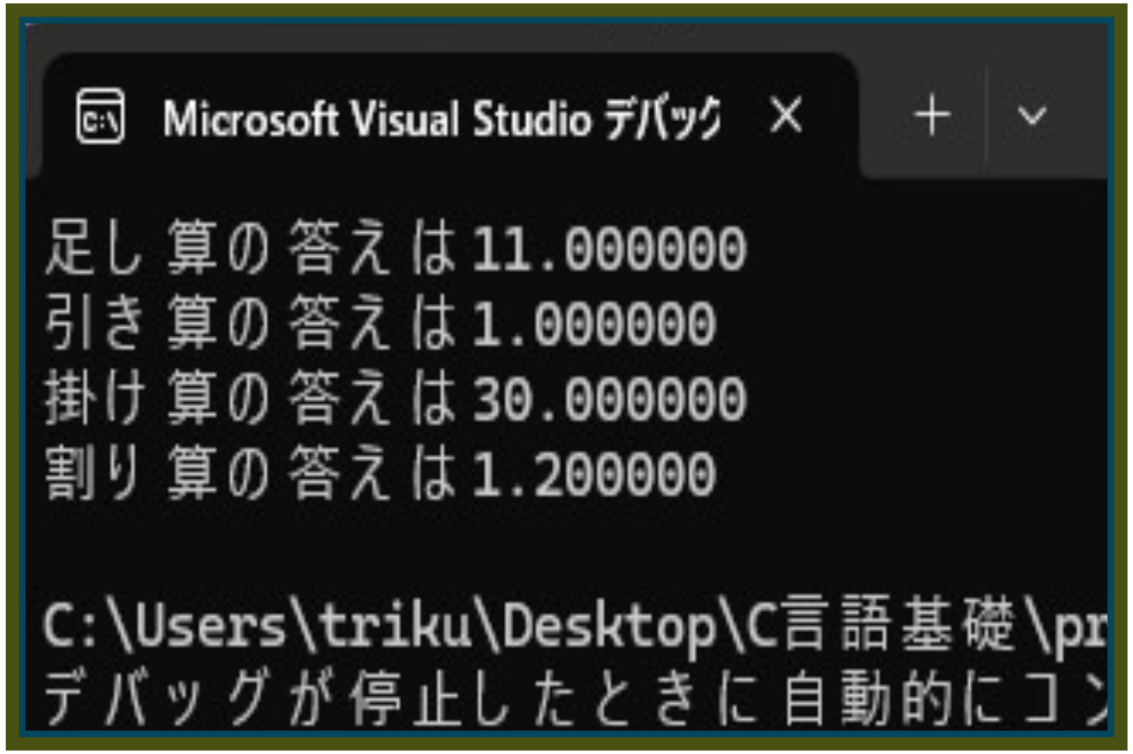
注意

float型には有効桁数というものがあり、小さすぎる数値は無視されます。

プログラム例

```
6   void main()
7   {
8       string addText = "足し算の答えは%f¥n";
9       string difText = "引き算の答えは%f¥n";
10      string mulText = "掛け算の答えは%f¥n";
11      string divText = "割り算の答えは%f¥n";
12
13      float firstValue = 6.0f;
14      float secondValue = 5.0f;
15
16      float add = firstValue + secondValue;
17      float dif = firstValue - secondValue;
18      float mul = firstValue * secondValue;
19      float div = firstValue / secondValue;
20
21      printf(addText.c_str(), add);
22      printf(difText.c_str(), dif);
23      printf(mulText.c_str(), mul);
24      printf(divText.c_str(), div);
25  }
```

実行結果



変数・四則演算

解説

一部よく使う計算が省略して書くことができるようになっています。
よく使うので見たらわかるようになりましょう。

記号	使い方	書き方	省略前
++	数値に１を足した数に上書きする	value++;	value = value + 1;
--	数値から１を引いた数に上書きする	value--;	value = value - 1;
+=	指定の値を足した数に上書きする	value += 2;	value = value + 2;
-=	指定の値を引いた数に上書きする	value -= 2;	value = value - 2;
*=	指定の値をかけた数に上書きする	value *= 2;	value = value * 2;
/=	指定の値を割った数に上書きする	value /= 2;	value =value / 2;